Resolución — Taller SQL sobre HR 7 — Operadores de conjuntos

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Ciudades con departamentos, sin repetir

Nivel: Básico · **Tablas:** `departments`, `locations`

Se pide. Listar las ciudades donde la empresa tiene al menos un departamento, sin repetir ninguna.

Mostrar: ciudad y código del país, ordenados por ciudad.

Idea. `DISTINCT` quita las filas repetidas del resultado. Un `JOIN` entre `hr.departments` y `hr.locations` repite la ciudad una vez por cada departamento que tiene.

Pasos.

1. Se une `hr.departments d` con `hr.locations l` por `location_id`.
2. Se seleccionan la ciudad y el país de la ubicación.
3. `SELECT DISTINCT` deja una sola fila por cada combinación distinta de ciudad y país.
4. Se ordena por ciudad.

```
SELECT DISTINCT l.city,
       l.country_id
FROM hr.departments d
JOIN hr.locations l ON l.location_id = d.location_id
ORDER BY l.city;
```

Resultado en HR. 7 filas y 2 columnas:

```
CITY                COUNTRY_ID
------------------  ----------
London              GB
Munich              DE
Oxford              GB
Seattle             US
South San Francisco US
Southlake           US
Toronto             CA
```

Ojo. Sin `DISTINCT` el resultado tendría 27 filas, una por departamento, y Seattle saldría 21 veces. `DISTINCT` se aplica a toda la fila seleccionada, no solo a la primera columna.

Variante. `GROUP BY` con las mismas columnas da el mismo resultado:

```
SELECT l.city,
       l.country_id
FROM hr.departments d
JOIN hr.locations l ON l.location_id = d.location_id
GROUP BY l.city, l.country_id
ORDER BY l.city;
```

Ejercicio 2 — Departamentos con más de un cargo

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar los departamentos donde trabajan empleados con más de un cargo distinto, con la cantidad de empleados y la cantidad de cargos.

Mostrar: nombre del departamento, empleados y cargos, de mayor a menor cantidad de cargos y, si empatan, por nombre del departamento.

Idea. `COUNT(DISTINCT col)` cuenta los valores distintos de una columna y no las filas.

Pasos.

1. Se unen `hr.employees` y `hr.departments` y se agrupa por departamento.
2. `COUNT(*)` cuenta empleados y `COUNT(DISTINCT e.job_id)` cuenta cargos distintos.
3. `HAVING COUNT(DISTINCT e.job_id) > 1` deja los departamentos con más de un cargo.
4. Se ordena por cargos descendente y por nombre.

```
SELECT d.department_name,
       COUNT(*) AS empleados,
       COUNT(DISTINCT e.job_id) AS cargos
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(DISTINCT e.job_id) > 1
ORDER BY cargos DESC, d.department_name;
```

Resultado en HR. 7 filas y 3 columnas:

```
DEPARTMENT_NAME  EMPLEADOS  CARGOS
---------------  ---------  ------
Shipping                45       3
Accounting               2       2
Executive                3       2
Finance                  6       2
Marketing                2       2
Purchasing               6       2
Sales                   34       2
```

Ojo. En Shipping hay 45 empleados pero solo 3 cargos distintos. Sin `DISTINCT`, `COUNT(e.job_id)` daría 45. `COUNT(DISTINCT ...)` ignora los `NULL`, igual que `COUNT(col)`. De los 11 departamentos con empleados, 7 tienen más de un cargo.

Ejercicio 3 — UNION frente a UNION ALL

Nivel: Intermedio · **Tablas:** `employees`, `job_history`

Se pide. Reunir los códigos de departamento que aparecen en `hr.employees` y en `hr.job_history` y comparar cuántas filas devuelven `UNION` y `UNION ALL`.

Mostrar: una sola fila con dos columnas: `con_union` y `con_union_all`.

Idea. `UNION` junta los resultados de dos consultas y elimina las filas repetidas. `UNION ALL` los junta sin eliminar nada.

Pasos.

1. Cada consulta devuelve una sola columna, `department_id`, para que las dos sean compatibles.
2. `UNION` quita los duplicados: queda cada departamento una vez.
3. `UNION ALL` conserva todas las filas: 107 de empleados más 10 del historial.
4. Cada unión se cuenta dentro de una vista en línea y las dos cuentas se piden desde `dual`.

```
SELECT (SELECT COUNT(*)
        FROM (SELECT e.department_id FROM hr.employees e
              UNION
              SELECT h.department_id FROM hr.job_history h) u)
         AS con_union,
       (SELECT COUNT(*)
        FROM (SELECT e.department_id FROM hr.employees e
              UNION ALL
              SELECT h.department_id FROM hr.job_history h) a)
         AS con_union_all
FROM dual;
```

Resultado en HR. 1 fila y 2 columnas:

```
CON_UNION  CON_UNION_ALL
---------  -------------
       12            117
```

Ojo. `UNION` trata los `NULL` como iguales: aunque hay una persona sin departamento, el `NULL` aparece una sola vez, y por eso el resultado es de 12 (los 11 departamentos más el `NULL`). `UNION ALL` da 117. Como `UNION` tiene que comparar filas para quitar repetidos, suele costar más: si ya se sabe que no hay repetidos, conviene `UNION ALL`.

Ejercicio 4 — Una fila de total con UNION ALL

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Mostrar la cantidad de empleados de cada departamento y, en la primera fila, una fila «TOTAL» que sume las demás.

Mostrar: departamento y cantidad de empleados, con la fila TOTAL primero y luego los departamentos de mayor a menor cantidad y, si empatan, por nombre.

Idea. `UNION ALL` permite pegar una fila calculada a otra consulta. Una columna auxiliar, `orden`, decide dónde queda cada parte.

Pasos.

1. La primera consulta cuenta empleados por departamento y le pone `orden` 1.
2. La segunda cuenta todos los empleados que tienen departamento, con el texto «TOTAL» y `orden` 0.
3. La unión se hace dentro de una vista en línea para poder ordenar por `orden` afuera.
4. El `ORDER BY` pone el total primero, y luego los departamentos por cantidad y nombre.

```
SELECT x.departamento,
       x.empleados
FROM (SELECT d.department_name AS departamento,
             COUNT(*) AS empleados,
             1 AS orden
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id
      GROUP BY d.department_name
      UNION ALL
      SELECT 'TOTAL',
             COUNT(e.department_id),
             0
      FROM hr.employees e) x
ORDER BY x.orden, x.empleados DESC, x.departamento;
```

Resultado en HR. 12 filas y 2 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO  EMPLEADOS
------------  ---------
TOTAL              106
Shipping            45
Sales               34
Finance              6
Purchasing           6
IT                   5
```

Ojo. Las dos consultas deben tener el mismo número de columnas y tipos compatibles; los nombres de columna salen de la primera. El total cuenta 106 porque `COUNT(e.department_id)` ignora a la persona sin departamento: con `COUNT(*)` saldría 107 y no coincidiría con la suma de las filas de arriba.

Ejercicio 5 — Departamentos con empleados y con historial: INTERSECT

Nivel: Intermedio · **Tablas:** `departments`, `employees`, `job_history`

Se pide. Listar los departamentos que tienen empleados hoy y que además aparecen en el historial de cargos.

Mostrar: código y nombre del departamento, ordenados por código.

Idea. `INTERSECT` devuelve solo las filas que están en las dos consultas, sin repetidos.

Pasos.

1. La primera consulta saca los `department_id` de `hr.employees` y la segunda los de `hr.job_history`.
2. `INTERSECT` se queda con los que están en las dos.
3. Se usa ese resultado como una lista en un `IN` para traer el nombre desde `hr.departments`.
4. Se ordena por código.

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE d.department_id IN (SELECT e.department_id FROM hr.employees e
                          INTERSECT
                          SELECT h.department_id FROM hr.job_history h)
ORDER BY d.department_id;
```

Resultado en HR. 6 filas y 2 columnas:

```
DEPARTMENT_ID  DEPARTMENT_NAME
-------------  ---------------
           20  Marketing
           50  Shipping
           60  IT
           80  Sales
           90  Executive
          110  Accounting
```

Ojo. `hr.job_history` tiene 10 filas pero solo seis departamentos distintos, y los seis tienen empleados hoy. `INTERSECT` también elimina los repetidos, como `UNION`.

Variante. Con dos `EXISTS` se llega al mismo resultado, pero hay que escribir cada condición:

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE EXISTS (SELECT 1
              FROM hr.employees e
              WHERE e.department_id = d.department_id)
  AND EXISTS (SELECT 1
              FROM hr.job_history h
              WHERE h.department_id = d.department_id)
ORDER BY d.department_id;
```

Ejercicio 6 — Departamentos sin empleados: MINUS y la trampa de NOT IN

Nivel: Avanzado · **Tablas:** `departments`, `employees`

Se pide. Listar los departamentos que no tienen ningún empleado, usando `MINUS`.

Mostrar: código y nombre del departamento, ordenados por código.

Idea. `MINUS` devuelve las filas de la primera consulta que no están en la segunda.

Pasos.

1. La primera consulta saca todos los `department_id` de `hr.departments`.
2. La segunda saca los `department_id` de `hr.employees`.
3. `MINUS` deja los departamentos que ningún empleado ocupa.
4. Ese resultado se usa en un `IN` para traer el nombre, y se ordena por código.

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE d.department_id IN (SELECT d2.department_id FROM hr.departments d2
                          MINUS
                          SELECT e.department_id FROM hr.employees e)
ORDER BY d.department_id;
```

Resultado en HR. 16 filas y 2 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_ID  DEPARTMENT_NAME
-------------  --------------------
          120  Treasury
          130  Corporate Tax
          140  Control And Credit
          150  Shareholder Services
          160  Benefits
          170  Manufacturing
```

Ojo. Es el mismo resultado del ejercicio 4 del taller 4: 16 departamentos. La versión con `NOT IN (SELECT e.department_id FROM hr.employees e)` devuelve cero filas, porque `department_id` tiene un `NULL` (Kimberely Grant) y `x NOT IN (..., NULL)` nunca es verdadero. `MINUS` compara los `NULL` como iguales y no tiene ese problema.

Variante. `NOT IN` funciona si la subconsulta descarta los nulos:

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE d.department_id NOT IN (SELECT e.department_id
                              FROM hr.employees e
                              WHERE e.department_id IS NOT NULL)
ORDER BY d.department_id;
```

Ejercicio 7 — Países con sedes pero sin departamentos

Nivel: Avanzado · **Tablas:** `countries`, `locations`, `departments`

Se pide. Listar los países que tienen ubicaciones registradas pero ninguna de ellas tiene departamentos.

Mostrar: código y nombre del país, ordenados por nombre.

Idea. La primera consulta saca los países con ubicaciones y la segunda los países con ubicaciones que sí tienen departamentos; `MINUS` deja los que están en la primera y no en la segunda.

Pasos.

1. Los países con ubicaciones salen de `hr.locations`.
2. Los países con departamentos salen de unir `hr.locations` con `hr.departments` por `location_id`.
3. `MINUS` resta los segundos de los primeros.
4. Se usa el resultado en un `IN` para traer el nombre del país y se ordena por nombre.

```
SELECT c.country_id,
       c.country_name
FROM hr.countries c
WHERE c.country_id IN (SELECT l.country_id FROM hr.locations l
                       MINUS
                       SELECT l2.country_id
                       FROM hr.locations l2
                       JOIN hr.departments d
                         ON d.location_id = l2.location_id)
ORDER BY c.country_name;
```

Resultado en HR. 10 filas y 2 columnas; se muestran las primeras 6 filas:

```
COUNTRY_ID  COUNTRY_NAME
----------  ------------
AU          Australia
BR          Brazil
CN          China
IN          India
IT          Italy
JP          Japan
```

Ojo. De los 14 países con ubicaciones, solo 4 (Estados Unidos, Canadá, Reino Unido y Alemania) tienen departamentos, por eso salen 10. Las dos consultas de un `MINUS` deben tener el mismo número de columnas y tipos compatibles.

Ejercicio 8 — Clasificar los países con UNION ALL

Nivel: Avanzado · **Tablas:** `countries`, `locations`, `departments`

Se pide. Clasificar los 25 países de la tabla en tres estados: «Con departamentos», «Solo con ubicaciones» y «Sin ubicaciones», y contar cuántos hay en cada estado.

Mostrar: estado y cantidad de países, de mayor a menor cantidad y, si empatan, por estado.

Idea. Cada estado es una consulta distinta con su propia etiqueta, y `UNION ALL` las reúne. Como cada país cae en un solo estado, no hay repetidos que quitar.

Pasos.

1. La primera consulta toma los países con alguna ubicación que tenga departamentos.
2. La segunda toma los países con ubicaciones, pero sin departamentos, con la resta de `MINUS`.
3. La tercera toma los países para los que no existe ninguna ubicación, con `NOT EXISTS`.
4. Una vista en línea reúne las tres y la consulta externa agrupa por estado.

```
SELECT x.estado,
       COUNT(*) AS paises
FROM (SELECT c.country_id,
             'Con departamentos' AS estado
      FROM hr.countries c
      WHERE c.country_id IN (SELECT l.country_id
                             FROM hr.locations l
                             JOIN hr.departments d
                               ON d.location_id = l.location_id)
      UNION ALL
      SELECT c.country_id,
             'Solo con ubicaciones'
      FROM hr.countries c
      WHERE c.country_id IN (SELECT l.country_id FROM hr.locations l
                             MINUS
                             SELECT l2.country_id
                             FROM hr.locations l2
                             JOIN hr.departments d
                               ON d.location_id = l2.location_id)
      UNION ALL
      SELECT c.country_id,
             'Sin ubicaciones'
      FROM hr.countries c
      WHERE NOT EXISTS (SELECT 1
                        FROM hr.locations l
                        WHERE l.country_id = c.country_id)) x
GROUP BY x.estado
ORDER BY paises DESC, x.estado;
```

Resultado en HR. 3 filas y 2 columnas:

```
ESTADO               PAISES
-------------------  ------
Sin ubicaciones          11
Solo con ubicaciones     10
Con departamentos         4
```

Ojo. Una buena comprobación de una clasificación es que las cuentas sumen el total: 11 + 10 + 4 = 25, el número de países. Si la suma no da, alguna condición se superpone o deja huecos. En la tercera consulta se usa `NOT EXISTS` y no `NOT IN`, para no depender de que no haya nulos.

Ejercicio 9 — Departamentos que contrataron en cada uno de tres años

Nivel: Avanzado · **Tablas:** `departments`, `employees`

Se pide. Listar los departamentos que contrataron al menos a una persona en cada uno de los años 2015, 2016 y 2017.

Mostrar: código y nombre del departamento, ordenados por código.

Idea. Es una división relacional: «para todos los años de la lista». Un departamento cumple si, al restar de la lista de años (2015, 2016 y 2017) los años en que contrató, no queda ninguno.

Pasos.

1. Una vista en línea arma la lista de tres años con `UNION ALL` sobre `dual`.
2. A esa lista se le resta, con `MINUS`, el conjunto de años en que contrató cada departamento.
3. `NOT EXISTS` pide que esa resta quede vacía: no falta ningún año.
4. Se ordena por código de departamento.

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE NOT EXISTS (SELECT y.anio
                  FROM (SELECT 2015 AS anio FROM dual
                        UNION ALL SELECT 2016 FROM dual
                        UNION ALL SELECT 2017 FROM dual) y
                  MINUS
                  SELECT EXTRACT(YEAR FROM e.hire_date)
                  FROM hr.employees e
                  WHERE e.department_id = d.department_id)
ORDER BY d.department_id;
```

Resultado en HR. 5 filas y 2 columnas:

```
DEPARTMENT_ID  DEPARTMENT_NAME
-------------  ---------------
           30  Purchasing
           50  Shipping
           60  IT
           80  Sales
          100  Finance
```

Ojo. Los 16 departamentos sin empleados no cumplen, porque la resta les deja los tres años. Salen cinco departamentos: Purchasing, Shipping, IT, Sales y Finance.

Variante. Contando años distintos se llega al mismo resultado con menos código, siempre que la lista de años sea corta y conocida:

```
SELECT d.department_id,
       d.department_name
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE EXTRACT(YEAR FROM e.hire_date) IN (2015, 2016, 2017)
GROUP BY d.department_id, d.department_name
HAVING COUNT(DISTINCT EXTRACT(YEAR FROM e.hire_date)) = 3
ORDER BY d.department_id;
```

Ejercicio 10 — Jefes de departamento que también dirigen personas

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar a quienes son jefe de un departamento (`hr.departments.manager_id`) y además son jefe directo de al menos un empleado (`hr.employees.manager_id`).

Mostrar: identificador, nombre y apellido, ordenados por apellido.

Idea. Son dos conjuntos de identificadores: los jefes de departamento y los jefes de personas. `INTERSECT` deja los que están en los dos.

Pasos.

1. La primera consulta saca `manager_id` de `hr.departments`.
2. La segunda saca `manager_id` de `hr.employees`.
3. `INTERSECT` deja los identificadores comunes.
4. Se usan en un `IN` para traer el nombre desde `hr.employees` y se ordena por apellido.

```
SELECT e.employee_id,
       e.first_name,
       e.last_name
FROM hr.employees e
WHERE e.employee_id IN (SELECT d.manager_id FROM hr.departments d
                        INTERSECT
                        SELECT s.manager_id FROM hr.employees s)
ORDER BY e.last_name;
```

Resultado en HR. 8 filas y 3 columnas:

```
EMPLOYEE_ID  FIRST_NAME  LAST_NAME
-----------  ----------  ---------
        121  Adam        Fripp
        108  Nancy       Gruenberg
        205  Shelley     Higgins
        103  Alexander   James
        100  Steven      King
        114  Den         Li
        201  Michael     Martinez
        145  John        Singh
```

Ojo. Las dos consultas traen `NULL` (departamentos sin jefe y el empleado sin jefe) y `INTERSECT` los trata como iguales, pero un `NULL` no coincide con ningún `employee_id` en el `IN`, así que no afecta al resultado. Hay 11 jefes de departamento y 8 de ellos dirigen personas.

Ejercicio 11 — Jefes de departamento que no dirigen a nadie

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar a quienes son jefe de un departamento pero no son jefe directo de ningún empleado.

Mostrar: identificador, nombre y apellido, ordenados por apellido.

Idea. Es el mismo par de conjuntos del ejercicio anterior, pero con `MINUS`: los jefes de departamento menos los jefes de personas.

Pasos.

1. La primera consulta saca `manager_id` de `hr.departments`.
2. La segunda saca `manager_id` de `hr.employees`.
3. `MINUS` deja los jefes de departamento que no aparecen como jefe de ningún empleado.
4. Se trae el nombre con un `IN` y se ordena por apellido.

```
SELECT e.employee_id,
       e.first_name,
       e.last_name
FROM hr.employees e
WHERE e.employee_id IN (SELECT d.manager_id FROM hr.departments d
                        MINUS
                        SELECT s.manager_id FROM hr.employees s)
ORDER BY e.last_name;
```

Resultado en HR. 3 filas y 3 columnas:

```
EMPLOYEE_ID  FIRST_NAME  LAST_NAME
-----------  ----------  ---------
        204  Hermann     Brown
        203  Susan       Jacobs
        200  Jennifer    Whalen
```

Ojo. Salen tres personas: Hermann Brown, Susan Jacobs y Jennifer Whalen, jefes de Public Relations, Human Resources y Administration, departamentos de una sola persona. Con el ejercicio anterior suman los 11 jefes de departamento: 8 más 3.

Ejercicio 12 — Reto: el rol de cada jefe

Nivel: Reto · **Tablas:** `departments`, `employees`

Se pide. Clasificar a todas las personas con alguna jefatura en tres roles: «Departamento y personas», «Solo departamento» y «Solo personas», y contar cuántas hay en cada rol.

Mostrar: rol y cantidad de personas, de mayor a menor cantidad y, si empatan, por rol.

Idea. Los tres roles salen de `INTERSECT` y de `MINUS` en los dos sentidos entre dos conjuntos. Una expresión `WITH` da nombre a cada conjunto para no repetir las consultas.

Pasos.

1. dep es el conjunto de jefes de departamento y `per` el de jefes de personas, sin `NULL`.
2. «Departamento y personas» es `dep INTERSECT per`.
3. «Solo departamento» es `dep MINUS per` y «Solo personas» es `per MINUS dep`.
4. `UNION ALL` reúne los tres con su etiqueta y la consulta externa cuenta por rol.

```
WITH dep AS (SELECT d.manager_id AS id
               FROM hr.departments d
               WHERE d.manager_id IS NOT NULL),
     per AS (SELECT s.manager_id AS id
               FROM hr.employees s
               WHERE s.manager_id IS NOT NULL)
SELECT x.rol,
       COUNT(*) AS personas
FROM (SELECT id, 'Departamento y personas' AS rol
      FROM (SELECT id FROM dep INTERSECT SELECT id FROM per)
      UNION ALL
      SELECT id, 'Solo departamento'
      FROM (SELECT id FROM dep MINUS SELECT id FROM per)
      UNION ALL
      SELECT id, 'Solo personas'
      FROM (SELECT id FROM per MINUS SELECT id FROM dep)) x
GROUP BY x.rol
ORDER BY personas DESC, x.rol;
```

Resultado en HR. 3 filas y 2 columnas:

```
ROL                     PERSONAS
----------------------  --------
Solo personas                 10
Departamento y personas        8
Solo departamento              3
```

Ojo. Salen 10, 8 y 3: en total 21 personas con alguna jefatura, que es lo que daría `UNION` de los dos conjuntos. Los tres roles no se superponen, por eso `UNION ALL` basta. Los `NULL` se quitan dentro de dep y `per` para que ninguno cuente como una persona más.